

Module 3: Data Manipulation

 **SQL Server Version:** This module has been converted from MySQL to SQL Server (T-SQL) syntax.

In this module, we will explore how to modify and manage data within database tables. While inserting data into tables is important, we often need to update existing records, remove unwanted data, or enforce rules that maintain data integrity. This module covers essential data manipulation operations and the constraints that help us maintain clean, consistent data.

1. Understanding Data Manipulation Operations

Data manipulation involves four core operations, commonly referred to as CRUD operations:

- **Create** - Insert new data (covered in previous modules)
- **Read** - Query and retrieve data (covered in previous modules)
- **Update** - Modify existing data
- **Delete** - Remove data

In this module, we focus primarily on the UPDATE and DELETE operations, along with techniques for removing all data from tables.

2. The UPDATE Statement

The UPDATE statement modifies existing records in a table. It allows you to change one or more column values for rows that match specific conditions.

2.1 Basic UPDATE Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

The WHERE clause is critical. Without it, all rows in the table will be updated, which is rarely what you want.

2.2 UPDATE Example with Single Column

Let's update the address for a student with roll number 5:

```
UPDATE students  
SET address = 'Pune'  
WHERE roll = 5;
```

Output:

```
Query OK, 1 row affected (0.02 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

This command locates the student with roll number 5 and updates their address to 'Pune'. Only one row is affected.

2.3 UPDATE Example with Multiple Columns

You can update multiple columns in a single UPDATE statement:

```
UPDATE students  
SET english = 70, maths = 80, address = 'Pune'  
WHERE roll = 5;
```

Output:

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

This updates three columns (english, maths, and address) for the student with roll number 5.

2.4 The Importance of the WHERE Clause

Consider what happens if you omit the WHERE clause:

```
UPDATE students
SET address = 'Mumbai';
```

Output:

```
Query OK, 450 rows affected (0.15 sec)
Rows matched: 450  Changed: 450  Warnings: 0
```

Warning: This updates the address for ALL students in the table. Always include a WHERE clause to target specific records unless you genuinely intend to update all rows.

2.5 Safe Update Practices in SQL Server

SQL Server doesn't have a "safe update mode" like MySQL, but you can use transactions to safely test updates before committing them. This allows you to preview changes and roll back if something goes wrong.

2.6 UPDATE with Calculations

You can also update columns based on calculations involving current values:

```
UPDATE students  
SET maths = maths + 5  
WHERE maths < 40;
```

Output:

```
Query OK, 23 rows affected (0.02 sec)  
Rows matched: 23  Changed: 23  Warnings: 0
```

This adds 5 marks to the maths score for all students who currently have less than 40 marks in maths.

3. The DELETE Statement

The DELETE statement removes rows from a table based on specified conditions. Like UPDATE, it requires careful use of the WHERE clause.

3.1 Basic DELETE Syntax

```
DELETE FROM table_name  
WHERE condition;
```

3.2 DELETE Example with WHERE Clause

Delete a specific student by roll number:

```
-- SQL Server does not have sql_safe_updates mode  
-- Use BEGIN TRANSACTION and ROLLBACK if needed for safety  
  
DELETE FROM students
```

```
WHERE roll = 5;
```

```
-- Re-enable safety: Use transactions or test with SELECT first
```

Output:

```
Query OK, 0 rows affected (0.00 sec)
```

```
Query OK, 1 row affected (0.01 sec)
```

```
Query OK, 0 rows affected (0.00 sec)
```

This removes the student record with roll number 5 from the students table.

3.3 DELETE Multiple Rows

You can delete multiple rows that match a condition:

```
DELETE FROM students  
WHERE maths < 35 AND science < 35;
```

Output:

```
Query OK, 8 rows affected (0.02 sec)
```

This deletes all students who have failed in both maths and science (scored less than 35 in both subjects).

3.4 DELETE All Rows

Omitting the WHERE clause will delete all rows in the table:

```
DELETE FROM students;
```

Output:

```
Query OK, 450 rows affected (0.25 sec)
```

Warning: This removes all data from the students table but keeps the table structure intact. This operation is logged row-by-row and can be slower for large tables.

4. The TRUNCATE Statement

TRUNCATE is a faster alternative to DELETE for removing all rows from a table. However, it has important differences from DELETE.

4.1 TRUNCATE Syntax

```
TRUNCATE TABLE table_name;
```

4.2 TRUNCATE Example

```
TRUNCATE TABLE students;
```

Output:

```
Query OK, 0 rows affected (0.05 sec)
```

This removes all data from the students table instantly by deallocating the data pages rather than deleting rows one by one.

4.3 DELETE vs TRUNCATE

Understanding the differences between DELETE and TRUNCATE is important for choosing the right operation:

Feature	DELETE	TRUNCATE
Row-by-row deletion	Yes	No (bulk deallocation)
WHERE clause support	Yes	No
Triggers fired	Yes	Varies by DBMS
Transaction rollback	All	Can be rolled back if in an uncommitted explicit transaction
Auto-increment reset	No	Yes (PostgreSQL, SQL Server)
Performance	Slower	Faster
Logs each row	Yes	No (logs page deallocation)

4.4 When to Use DELETE vs TRUNCATE

Use DELETE when:

- You need to remove specific rows based on conditions
- You need to preserve auto-increment values
- You need triggers to fire for each deleted row
- You need detailed transaction logging

Use TRUNCATE when:

- You want to remove all rows from a table
- You want better performance for large tables

- You want to reset auto-increment counters
- You don't need row-level logging

5. Database Constraints

Constraints are rules enforced on database tables to ensure data integrity and accuracy. They prevent invalid data from being entered into the database and maintain relationships between tables.

5.1 Types of Constraints

SQL supports several types of constraints:

- **PRIMARY KEY** - Uniquely identifies each record in a table
- **FOREIGN KEY** - Establishes relationships between tables
- **UNIQUE** - Ensures all values in a column are different
- **NOT NULL** - Ensures a column cannot have NULL values
- **CHECK** - Validates data based on a condition
- **DEFAULT** - Provides a default value when no value is specified

6. PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies each row in a table. A table can have only one primary key, and it enforces two rules:

- No duplicate values are allowed
- NULL values are not permitted

6.1 Creating a Table with PRIMARY KEY

```
CREATE TABLE employees
(  
    eid INT PRIMARY KEY,  
    ename VARCHAR(30) NOT NULL,
```



```
email VARCHAR(100) NOT NULL UNIQUE,  
esalary DECIMAL(10, 2) DEFAULT 15000 CHECK(esalary >= 15000)  
);
```

Output:

Query OK, 0 rows affected (0.08 sec)

Here, the `eid` column is designated as the primary key. This means each employee must have a unique employee ID, and the ID cannot be NULL.

6.2 Testing PRIMARY KEY - Valid Insert

```
INSERT INTO employees(eid, ename, email, esalary)  
VALUES(1, 'Arun', '[email protected]', 45000);
```

Output:

Query OK, 1 row affected (0.01 sec)

This insert succeeds because the employee ID is unique and not NULL.

6.3 Testing PRIMARY KEY - Duplicate Value Error

```
INSERT INTO employees(eid, ename, email, esalary)  
VALUES(1, 'Nidhi', '[email protected]', 60000);
```

Output:

```
ERROR 1062 (23000): Duplicate entry '1' for key 'employees.PRIMARY'
```

This insert fails because employee ID 1 already exists. The PRIMARY KEY constraint prevents duplicate values.

6.4 Testing PRIMARY KEY - NULL Value Error

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(NULL, 'Nidhi', '[email protected]', 60000);
```

Output:

```
ERROR 1048 (23000): Column 'eid' cannot be null
```

This insert fails because the primary key column cannot accept NULL values.

6.5 Key Characteristics of PRIMARY KEY

- Each table can have only one PRIMARY KEY
- The PRIMARY KEY uniquely identifies each row
- PRIMARY KEY automatically creates an index for faster data retrieval
- Cannot contain duplicate or NULL values
- Often used in relationships with other tables through FOREIGN KEYS

7. NOT NULL Constraint

The NOT NULL constraint ensures that a column must always contain a value. It prevents NULL (empty or unknown) values from being inserted.

7.1 Testing NOT NULL Constraint

```
INSERT INTO employees(eid, email, esalary)
VALUES(2, '[email protected]', 50000);
```

Output:

```
ERROR 1364 (HY000): Field 'ename' doesn't have a default value
```

This insert fails because the `ename` column has a NOT NULL constraint, and no value was provided for it.

7.2 Correct Usage with NOT NULL

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(2, 'Vidhi', '[email protected]', 50000);
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

This insert succeeds because all NOT NULL columns now have values.

8. UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are different from each other. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints, and UNIQUE columns can contain NULL values (typically only one NULL is allowed).

8.1 Testing UNIQUE Constraint

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(3, 'Arvind', '[email protected]', 55000);
```

Output:

```
ERROR 1062 (23000): Duplicate entry '[email protected]' for key 'employees.email'
```

This insert fails because the email '' already exists in the table. The UNIQUE constraint on the email column prevents duplicate email addresses.

8.2 Correct Usage with UNIQUE

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(3, 'Arvind', '[email protected]', 55000);
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

This succeeds because '' is unique.

9. DEFAULT Constraint

The DEFAULT constraint provides a default value for a column when no value is specified during INSERT operations.

9.1 Testing DEFAULT Constraint

```
INSERT INTO employees(eid, ename, email)
VALUES(4, 'Vidhi', '[email protected]');
```

Output:

Query OK, 1 row affected (0.01 sec)

Notice that we did not provide a value for `esalary`. Let's verify what value was inserted:

```
SELECT * FROM employees WHERE eid = 4;
```

Output:

```
+-----+-----+-----+-----+
| eid | ename | email          | esalary |
+-----+-----+-----+-----+
|  4  | Vidhi | [email protected] | 15000.00|
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

The salary was automatically set to 15000.00, which is the default value specified in the table definition. This is useful for columns where you have a standard or common value.

10. CHECK Constraint

The CHECK constraint validates that values in a column satisfy a specific condition. It ensures that only data meeting certain criteria can be entered.

10.1 Testing CHECK Constraint - Violation

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(5, 'Amit', '[email protected]', 5000);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'employees_chk_1' is violated.
```

This insert fails because the salary of 5000 does not satisfy the CHECK constraint `esalary >= 15000`. The database enforces that all employees must have a salary of at least 15000.

10.2 Correct Usage with CHECK

```
INSERT INTO employees(eid, ename, email, esalary)
VALUES(5, 'Amit', '[email protected]', 25000);
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

This succeeds because 25000 meets the minimum salary requirement.

10.3 Advanced CHECK Constraints

CHECK constraints can include more complex conditions:

```
CREATE TABLE products
(
    pid INT PRIMARY KEY,
    pname VARCHAR(20) NOT NULL,
```

```
price DECIMAL(10, 2) CHECK(price >= 10) DEFAULT 10,  
p_cat VARCHAR(30)  
    CHECK(p_cat IN('Foods', 'Stationary', 'Electronics')),  
p_weight DECIMAL(5, 2) CHECK(p_weight BETWEEN 1 AND 500),  
p_barcode VARCHAR(20) UNIQUE NOT NULL  
);
```

Output:

Query OK, 0 rows affected (0.07 sec)

This products table demonstrates multiple CHECK constraints:

- `price >= 10` ensures minimum price
- `p_cat IN('Foods', 'Stationary', 'Electronics')` restricts category to specific values
- `p_weight BETWEEN 1 AND 500` ensures weight is within a valid range

11. FOREIGN KEY Constraint

The FOREIGN KEY constraint establishes a link between two tables. It ensures referential integrity by requiring that values in one table match values in another table's primary key or unique column.

11.1 Creating Tables with FOREIGN KEY

First, create a parent table (departments):

```
CREATE TABLE departments  
(  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

Output:

Query OK, 0 rows affected (0.06 sec)

Now create a child table (staff) that references departments:

```
CREATE TABLE staff
(
    staff_id INT PRIMARY KEY,
    staff_name VARCHAR(50) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Output:

Query OK, 0 rows affected (0.08 sec)

The FOREIGN KEY constraint on `dept_id` ensures that any department ID entered in the staff table must exist in the departments table.

11.2 Testing FOREIGN KEY - Valid Insert

```
INSERT INTO departments(dept_id, dept_name)
VALUES(1, 'Human Resources');

INSERT INTO staff(staff_id, staff_name, dept_id)
VALUES(101, 'Rajesh Kumar', 1);
```

Output:

Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.01 sec)

Both inserts succeed because department 1 exists before we try to assign it to a staff member.

11.3 Testing FOREIGN KEY - Violation

```
INSERT INTO staff(staff_id, staff_name, dept_id)
VALUES(102, 'Priya Sharma', 5);
```

Output:

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails

This insert fails because department ID 5 does not exist in the departments table. The FOREIGN KEY constraint maintains referential integrity.

11.4 FOREIGN KEY Benefits

- Maintains referential integrity between related tables
- Prevents orphaned records (records that reference non-existent data)
- Can enforce cascading actions (CASCADE DELETE, CASCADE UPDATE)
- Documents relationships between tables

12. Constraint Summary

Here's a comprehensive summary of all constraint types:

Constraint	Purpose	Key Features
PRIMARY KEY	Uniquely identifies each record	No duplicates, no NULLs, only one per table
FOREIGN KEY	Links tables together	Maintains referential integrity
UNIQUE	Ensures unique values	No duplicates, allows NULL (usually one)
NOT NULL	Requires a value	Prevents NULL values
CHECK	Validates data against conditions	Custom logical conditions
DEFAULT	Provides default values	Used when no value is specified

13. Practical Application Example

Let's create a comprehensive example that uses multiple constraints together:

```
CREATE TABLE customer_orders
(
  order_id INT PRIMARY KEY IDENTITY(1,1),
  customer_id INT NOT NULL,
  order_date DATE NOT NULL DEFAULT CURRENT_DATE,
  total_amount DECIMAL(10, 2) CHECK(total_amount > 0),
  order_status VARCHAR(20)
    CHECK(order_status IN('Pending', 'Shipped', 'Delivered', 'Cancelled')),
  delivery_email VARCHAR(100) UNIQUE,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Output:

Query OK, 0 rows affected (0.09 sec)

This table demonstrates:

- **PRIMARY KEY** on order_id with IDENTITY(1,1)
- **NOT NULL** on customer_id and order_date (required fields)
- **DEFAULT** value for order_date (current date if not specified)
- **CHECK** constraints on total_amount (must be positive) and order_status (limited to specific values)
- **UNIQUE** on delivery_email (no duplicate delivery emails)
- **FOREIGN KEY** linking to customers table

14. Best Practices for Data Manipulation

14.1 UPDATE Best Practices

- Always use a WHERE clause unless you intentionally want to update all rows
- Test UPDATE statements with SELECT first to verify which rows will be affected
- Use transactions for critical updates to allow rollback if needed
- Be cautious when updating primary or foreign key columns

14.2 DELETE Best Practices

- Always include a WHERE clause to avoid deleting all data
- Consider using soft deletes (marking records as deleted) instead of hard deletes for important data
- Verify data before deletion with a SELECT statement
- Be aware of FOREIGN KEY constraints that might prevent deletion

14.3 Constraint Best Practices

- Define PRIMARY KEY for every table to uniquely identify records
- Use FOREIGN KEY constraints to maintain referential integrity

- Apply NOT NULL to columns that should always have values
- Use CHECK constraints to validate data at the database level
- Document constraint rules clearly for other developers

15. Common Pitfalls and How to Avoid Them

15.1 Forgetting the WHERE Clause

The most common mistake is omitting the WHERE clause in UPDATE or DELETE statements. Always double-check your statements before execution.

15.2 Constraint Violations During Bulk Operations

When inserting or updating multiple records, a single constraint violation can cause the entire operation to fail. Consider validating data before performing bulk operations.

15.3 Circular Foreign Key References

Be careful when creating foreign keys between tables that reference each other. This can create insertion challenges. Plan your table relationships carefully and consider using NULL-able foreign keys or deferred constraints where appropriate.

15.4 Ignoring Index Performance

PRIMARY KEY and UNIQUE constraints automatically create indexes, but you may need additional indexes for FOREIGN KEY columns to optimize join performance.

15.5 Over-constraining Data

While constraints are valuable, too many constraints can make data entry difficult and slow down INSERT operations. Balance data integrity with practical usability.

16. ALTER TABLE Operations

The ALTER TABLE statement is one of the most powerful DDL (Data Definition Language) commands. It allows you to modify existing table structures without losing data. This includes adding or removing columns, changing data types, renaming objects, and managing constraints.

Important: Different database systems have varying syntax for ALTER TABLE operations. This section focuses on SQL Server but includes comparison tables showing equivalent operations in MySQL, Oracle, and PostgreSQL.

16.1 Adding a Column

To add a new column to an existing table:

```
-- SQL Server
ALTER TABLE employees
ADD phone_number VARCHAR(15);
```

Output:

```
Command completed successfully.
```

Adding a column with constraints:

```
ALTER TABLE employees
ADD date_of_birth DATE NOT NULL DEFAULT '1990-01-01';
```

You can also add multiple columns in a single statement:

```
ALTER TABLE employees
ADD
    address VARCHAR(200),
    city VARCHAR(50),
    postal_code VARCHAR(10);
```

16.2 Removing (Dropping) a Column

To remove a column from a table:

```
-- SQL Server
ALTER TABLE employees
DROP COLUMN phone_number;
```

Output:

Command completed successfully.

Warning: Dropping a column permanently deletes all data in that column. This operation cannot be undone.

16.3 Renaming a Column

SQL Server uses a stored procedure to rename columns:

```
-- SQL Server
EXEC sp_rename 'employees.ename', 'employee_name', 'COLUMN';
```

Output:

Caution: Changing any part of an object name could break scripts and stored procedures.

16.4 Renaming a Table

To rename an entire table:

```
-- SQL Server  
EXEC sp_rename 'employees', 'staff_members';
```

Output:

Caution: Changing any part of an object name could break scripts and stored procedures.

16.5 Changing Column Data Type

To modify the data type of an existing column:

```
-- SQL Server  
ALTER TABLE employees  
ALTER COLUMN esalary DECIMAL(12, 2);
```

Output:

Command completed successfully.

Important Considerations:

- Ensure the existing data is compatible with the new data type

- Changing from a larger to smaller type may cause data loss
- Some type conversions may not be allowed if data exists

16.6 Adding Constraints

16.6.1 Adding PRIMARY KEY

```
-- SQL Server
ALTER TABLE departments
ADD CONSTRAINT pk_departments PRIMARY KEY (dept_id);
```

16.6.2 Adding FOREIGN KEY

```
-- SQL Server
ALTER TABLE employees
ADD CONSTRAINT fk_emp_dept
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id);
```

16.6.3 Adding UNIQUE Constraint

```
-- SQL Server
ALTER TABLE employees
ADD CONSTRAINT uq_emp_email UNIQUE (email);
```

16.6.4 Adding CHECK Constraint

```
-- SQL Server
ALTER TABLE employees
ADD CONSTRAINT chk_salary CHECK (esalary >= 15000);
```


16.6.5 Adding DEFAULT Constraint

```
-- SQL Server
ALTER TABLE employees
ADD CONSTRAINT df_salary DEFAULT 15000 FOR esalary;
```

16.6.6 Adding NOT NULL Constraint

In SQL Server, you modify the column definition to add NOT NULL:

```
-- First ensure there are no NULL values
UPDATE employees SET email = '[email protected]' WHERE email IS NULL;

-- Then alter the column
ALTER TABLE employees
ALTER COLUMN email VARCHAR(100) NOT NULL;
```

16.7 Removing (Dropping) Constraints

16.7.1 Dropping PRIMARY KEY

```
-- SQL Server
ALTER TABLE departments
DROP CONSTRAINT pk_departments;
```

16.7.2 Dropping FOREIGN KEY

```
-- SQL Server
ALTER TABLE employees
DROP CONSTRAINT fk_emp_dept;
```

16.7.3 Dropping UNIQUE Constraint

```
-- SQL Server
ALTER TABLE employees
DROP CONSTRAINT uq_emp_email;
```

16.7.4 Dropping CHECK Constraint

```
-- SQL Server
ALTER TABLE employees
DROP CONSTRAINT chk_salary;
```

16.7.5 Dropping DEFAULT Constraint

```
-- SQL Server
ALTER TABLE employees
DROP CONSTRAINT df_salary;
```

16.7.6 Removing NOT NULL Constraint

To allow NULL values in a column that was previously NOT NULL:

```
-- SQL Server
ALTER TABLE employees
ALTER COLUMN email VARCHAR(100) NULL;
```

16.8 Finding Constraint Names

Often you need to find the system-generated constraint names before dropping them:

```
-- Find all constraints for a table
SELECT
    CONSTRAINT_NAME,
    CONSTRAINT_TYPE,
    TABLE_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
```

```
WHERE TABLE_NAME = 'employees';

-- Find constraint details
SELECT
    CON.CONSTRAINT_NAME,
    CON.CONSTRAINT_TYPE,
    COL.COLUMN_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS CON
INNER JOIN INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE COL
    ON CON.CONSTRAINT_NAME = COL.CONSTRAINT_NAME
WHERE CON.TABLE_NAME = 'employees';
```

17. Cross-Database ALTER TABLE Comparison

Different database systems use varying syntax for ALTER TABLE operations. Below is a comprehensive comparison showing how to perform common operations across SQL Server, MySQL, Oracle, and PostgreSQL.

17.1 Adding a Column

Database	Syntax
SQL Server	<code>ALTER TABLE employees ADD phone VARCHAR(15);</code>
MySQL	<code>ALTER TABLE employees ADD phone VARCHAR(15);</code>
Oracle	<code>ALTER TABLE employees ADD (phone VARCHAR2(15));</code>
PostgreSQL	<code>ALTER TABLE employees ADD COLUMN phone VARCHAR(15);</code>

17.2 Removing (Dropping) a Column

Database	Syntax
SQL Server	<code>ALTER TABLE employees DROP COLUMN phone;</code>

Database	Syntax
MySQL	<code>ALTER TABLE employees DROP COLUMN phone;</code>
Oracle	<code>ALTER TABLE employees DROP COLUMN phone;</code>
PostgreSQL	<code>ALTER TABLE employees DROP COLUMN phone;</code>

17.3 Renaming a Column

Database	Syntax
SQL Server	<code>EXEC sp_rename 'employees.ename', 'emp_name', 'COLUMN';</code>
MySQL	<code>ALTER TABLE employees RENAME COLUMN ename TO emp_name;</code> Or: <code>ALTER TABLE employees CHANGE ename emp_name VARCHAR(50);</code>
Oracle	<code>ALTER TABLE employees RENAME COLUMN ename TO emp_name;</code>
PostgreSQL	<code>ALTER TABLE employees RENAME COLUMN ename TO emp_name;</code>

17.4 Renaming a Table

Database	Syntax
SQL Server	<code>EXEC sp_rename 'employees', 'staff';</code>
MySQL	<code>RENAME TABLE employees TO staff;</code> Or: <code>ALTER TABLE employees RENAME TO staff;</code>
Oracle	<code>RENAME employees TO staff;</code> Or: <code>ALTER TABLE employees RENAME TO staff;</code>

Database	Syntax
PostgreSQL	<code>ALTER TABLE employees RENAME TO staff;</code>

17.5 Changing Column Data Type

Database	Syntax
SQL Server	<code>ALTER TABLE employees ALTER COLUMN esalary DECIMAL(12,2);</code>
MySQL	<code>ALTER TABLE employees MODIFY esalary DECIMAL(12,2);</code> Or: <code>ALTER TABLE employees CHANGE esalary esalary DECIMAL(12,2);</code>
Oracle	<code>ALTER TABLE employees MODIFY (esalary NUMBER(12,2));</code>
PostgreSQL	<code>ALTER TABLE employees ALTER COLUMN esalary TYPE DECIMAL(12,2);</code>

17.6 Adding PRIMARY KEY Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees ADD CONSTRAINT pk_emp PRIMARY KEY (eid);</code>
MySQL	<code>ALTER TABLE employees ADD CONSTRAINT pk_emp PRIMARY KEY (eid);</code> Or: <code>ALTER TABLE employees ADD PRIMARY KEY (eid);</code>
Oracle	<code>ALTER TABLE employees ADD CONSTRAINT pk_emp PRIMARY KEY (eid);</code>

Database	Syntax
PostgreSQL	<code>ALTER TABLE employees ADD CONSTRAINT pk_emp PRIMARY KEY (eid);</code>

17.7 Dropping PRIMARY KEY Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees DROP CONSTRAINT pk_emp;</code>
MySQL	<code>ALTER TABLE employees DROP PRIMARY KEY;</code>
Oracle	<code>ALTER TABLE employees DROP CONSTRAINT pk_emp;</code> Or: <code>ALTER TABLE employees DROP PRIMARY KEY;</code>
PostgreSQL	<code>ALTER TABLE employees DROP CONSTRAINT pk_emp;</code>

17.8 Adding FOREIGN KEY Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id);</code>
MySQL	<code>ALTER TABLE employees ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id);</code>
Oracle	<code>ALTER TABLE employees ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id);</code>
PostgreSQL	

Database	Syntax
	<pre>ALTER TABLE employees ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id);</pre>

17.9 Dropping FOREIGN KEY Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees DROP CONSTRAINT fk_dept;</pre>
MySQL	<pre>ALTER TABLE employees DROP FOREIGN KEY fk_dept;</pre>
Oracle	<pre>ALTER TABLE employees DROP CONSTRAINT fk_dept;</pre>
PostgreSQL	<pre>ALTER TABLE employees DROP CONSTRAINT fk_dept;</pre>

17.10 Adding UNIQUE Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);</pre>
MySQL	<pre>ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);</pre> Or: <pre>ALTER TABLE employees ADD UNIQUE (email);</pre>
Oracle	<pre>ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);</pre>
PostgreSQL	

Database	Syntax
	<pre>ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);</pre>

17.11 Dropping UNIQUE Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees DROP CONSTRAINT uq_email;</pre>
MySQL	<pre>ALTER TABLE employees DROP INDEX uq_email;</pre>
Oracle	<pre>ALTER TABLE employees DROP CONSTRAINT uq_email; Or: ALTER TABLE employees DROP UNIQUE (email);</pre>
PostgreSQL	<pre>ALTER TABLE employees DROP CONSTRAINT uq_email;</pre>

17.12 Adding CHECK Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (esalary >= 15000);</pre>
MySQL	<pre>ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (esalary >= 15000);</pre>
Oracle	<pre>ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (esalary >= 15000);</pre>

Database	Syntax
PostgreSQL	<pre>ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (esalary >= 15000);</pre>

17.13 Dropping CHECK Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees DROP CONSTRAINT chk_salary;</pre>
MySQL	<pre>ALTER TABLE employees DROP CHECK chk_salary;</pre>
Oracle	<pre>ALTER TABLE employees DROP CONSTRAINT chk_salary;</pre>
PostgreSQL	<pre>ALTER TABLE employees DROP CONSTRAINT chk_salary;</pre>

17.14 Adding DEFAULT Constraint

Database	Syntax
SQL Server	<pre>ALTER TABLE employees ADD CONSTRAINT df_salary DEFAULT 15000 FOR esalary;</pre>
MySQL	<pre>ALTER TABLE employees ALTER esalary SET DEFAULT 15000;</pre>
Oracle	<pre>ALTER TABLE employees MODIFY (esalary DEFAULT 15000);</pre>
PostgreSQL	<pre>ALTER TABLE employees ALTER COLUMN esalary SET DEFAULT 15000;</pre>

17.15 Dropping DEFAULT Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees DROP CONSTRAINT df_salary;</code>
MySQL	<code>ALTER TABLE employees ALTER esalary DROP DEFAULT;</code>
Oracle	<code>ALTER TABLE employees MODIFY (esalary DEFAULT NULL);</code>
PostgreSQL	<code>ALTER TABLE employees ALTER COLUMN esalary DROP DEFAULT;</code>

17.16 Adding NOT NULL Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees ALTER COLUMN email VARCHAR(100) NOT NULL;</code>
MySQL	<code>ALTER TABLE employees MODIFY email VARCHAR(100) NOT NULL;</code>
Oracle	<code>ALTER TABLE employees MODIFY (email VARCHAR2(100) NOT NULL);</code>
PostgreSQL	<code>ALTER TABLE employees ALTER COLUMN email SET NOT NULL;</code>

17.17 Dropping NOT NULL Constraint

Database	Syntax
SQL Server	<code>ALTER TABLE employees ALTER COLUMN email VARCHAR(100) NULL;</code>

Database	Syntax
MySQL	<code>ALTER TABLE employees MODIFY email VARCHAR(100) NULL;</code>
Oracle	<code>ALTER TABLE employees MODIFY (email VARCHAR2(100) NULL);</code>
PostgreSQL	<code>ALTER TABLE employees ALTER COLUMN email DROP NOT NULL;</code>

18. Summary and Key Takeaways

This module covered the essential operations for manipulating data and table structures in SQL Server:

18.1 Data Manipulation Commands

- **UPDATE** - Modify existing records in a table
- **DELETE** - Remove records from a table
- Always use WHERE clauses to target specific rows
- Test operations with SELECT before executing

18.2 Constraints for Data Integrity

- **PRIMARY KEY** - Uniquely identifies each record
- **FOREIGN KEY** - Maintains relationships between tables
- **UNIQUE** - Ensures no duplicate values
- **NOT NULL** - Requires a value in the column
- **CHECK** - Validates data against custom conditions
- **DEFAULT** - Provides automatic values when none specified

18.3 Table Structure Modifications

- **ALTER TABLE** provides powerful capabilities for modifying existing tables

- Add, remove, or modify columns without losing data
- Add or remove constraints as requirements change
- Rename columns and tables for better organization
- Different database systems have varying syntax - always consult documentation

18.4 Best Practices

- Plan your database schema carefully before implementation
- Use constraints to enforce business rules at the database level
- Always backup data before major ALTER TABLE operations
- Test DDL statements in a development environment first
- Document all constraints and their purposes
- Consider the impact on existing applications when modifying table structures

Next Steps: In the next module, we will explore advanced querying techniques including JOINS, subqueries, and aggregate functions to retrieve and analyze data across multiple tables.